49. Packet Filtering Mental Model (nftables)

49. Packet Filtering Mental Model (nftables)

Phase: Phase 6 — Networking
Estimated time: 1–2 days
Slug: `49-networking-nft`

Learning objectives

- Explain chains/tables conceptually
- Prefer high-level tools when enough (ufw/firewalld)
- Log drops thoughtfully
- Test rules remotely with care

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

A firewall decides which packets reach your services. A good one makes a compromised app far less useful to an attacker; a bad one locks you out of your own server at the worst moment. nftables is the modern Linux packet filter in the kernel. Tools like ufw and firewalld are front ends that generate nftables (or iptables-nft) rules, so the mental model carries over.

The principle that matters most is the default policy for inbound traffic: **deny**. You list what is allowed (SSH, HTTPS) and everything else is dropped. The opposite, default allow with a few blocks, fails open: every new service you start is instantly exposed.

Core concepts

Term	Meaning
Table	container for chains, tied to a family (`inet`, `ip`, `ip6`)
Chain	ordered list of rules at a hook (`input`, `forward`, `output`)
Rule	match plus verdict (`accept`, `drop`, `reject`)
Policy	verdict when no rule matches; set to `drop` for input
State	`ct state established,related` lets replies back in
Set	named collection of addresses or ports
Counter / log	observe what a rule matches

```
  packet in --> [ prerouting ] --> routing decision
                                     |-> local? --> [ input chain ]  --> socket
                                     |              policy drop
                                     |-> other? --> [ forward chain ]
  app out ----> [ output chain ] ---> network
  input order: lo accept -> established accept -> ssh accept -> https accept ->
(drop)
```

Rules evaluate top to bottom, first terminal verdict wins. Order matters, especially “established” and loopback rules at the top.

Worked examples

Example 1: a minimal default-deny host firewall.

```
# /etc/nftables.conf
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    iif lo accept
    ct state established,related accept
    ct state invalid drop
    ip protocol icmp accept
    ip6 nexthdr icmpv6 accept
    tcp dport 22 accept
    tcp dport { 80, 443 } accept
  }
  chain forward { type filter hook forward priority 0; policy drop; }
  chain output  { type filter hook output priority 0; policy accept; }
}
```

Example 2: load safely and inspect.

```
sudo nft -c -f /etc/nftables.conf     # syntax check only
sudo nft -f /etc/nftables.conf
sudo nft list ruleset
```

Example 3: the same intent with ufw.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Example 4: add a counter and a rate-limited log to see drops.

```
sudo nft add rule inet filter input limit rate 5/minute log prefix "nft-drop: "
counter drop
journalctl -k | grep nft-drop | tail
```

Example 5: test from outside, not from the box itself.

```
nc -vz -w 2 server.example.com 22
nc -vz -w 2 server.example.com 5432
```

Open port 22 should connect; 5432 should time out (dropped) or be refused.

Mini-lab

1. In a VM with console access, take a snapshot first.
2. Run `sudo nft list ruleset` and save it to a file as a baseline.
3. Install the Example 1 rules, check with `nft -c`, then load.
4. Keep a second SSH session open. Verify you can open a new one.
5. Start `python3 -m http.server 8080` and test from another machine; the port should not be reachable because only 22, 80, and 443 are allowed.
6. Add `tcp dport 8080 accept`, test again, and then remove it.
7. Safety net: schedule `sleep 120; sudo nft flush ruleset` in another terminal before experimenting remotely, then cancel it when satisfied.

Decision guide / failure modes

Situation	Choice
Single server, simple needs	ufw or firewalld
Complex sets, NAT, custom logic	nftables directly
Container host	know that Docker adds its own rules; check `nft list ruleset`
Remote change	schedule a revert or use a console
Debugging drops	counters and rate-limited log

Failure modes: policy accept on input; forgetting the established rule so replies die; no loopback rule so local services break; applying rules over SSH with no rollback; mixing ufw, firewalld, and raw nft that overwrite each other; and never persisting rules so a reboot removes them (or loads permissive defaults). Verify after reboot.

Common mistakes

- Locking yourself out mid-SSH
- Logging every accepted packet
- Contradictory overlapping rules

Exercises

Work in `exercises/49-networking-nft/`. Read that folder's `README.md` fully before coding.

```
cd exercises/49-networking-nft
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `firewall_default_inbound`. It prints the single lowercase word naming the default inbound policy the lesson recommends for a server firewall. Nothing else is printed and nothing on the system is touched; the test is a recall check, not a rule load. After it passes, work the mini-lab only inside a VM snapshot with console access. Record the output of `nft list ruleset` before and after your change in your notes, and write the exact three rules that must appear above any `drop` so you do not lose SSH or break replies. Never test an unfamiliar ruleset on a remote-only machine.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/49-networking-nft/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Why allow established?
2. How avoid lockout?
3. ufw vs raw nftables?

Further reading

- https://wiki.nftables.org/